Phase 2 — Week 2 Project Assignment

AI-Powered Test Case Generator — User Story to Structured JSON

Goal

Build a script that reads a user story from a `.txt` file, sends it to the Gemini API with a carefully crafted system prompt instructing it to act as a QA engineer, receives structured JSON test cases back, prints them to the terminal in a formatted report, and saves them to a `.json` file — exactly mirroring what your final ADO/Jira pipeline will do.

Step by Step — What Your Script Must Do

STEP 1 — Load API key safely from .env file **Day 13**

- Use `python-dotenv` to load your Gemini API key from a `.env` file
- Never hardcode the key in your script
- If the key is missing or empty, raise a custom exception `MissingAPIKey` and exit cleanly

STEP 2 — Read user story from a .txt file **Phase 1 Week 3 reused**

- Read `user_story.txt` from disk using file handling
- Strip extra whitespace
- If the file is empty or missing, raise a custom exception `EmptyUserStory` and exit cleanly

STEP 3 — Build the prompt **Day 11 — System Prompt**

- Define a `system_prompt` string that gives Gemini its QA role
- Instruct it to return raw JSON only — no markdown, no explanation, no extra text
- Specify the exact JSON structure Gemini must follow (see Expected AI Output below)
- Build a `user_prompt` that injects the user story text

STEP 4 — Call Gemini API with controlled parameters **Day 9 + 10**

- Use `google-generativeai` SDK
- Set `temperature=0.2` — low for consistent, precise test cases
- Set `max_output_tokens=1000`
- Send both system prompt and user prompt
- Wrap the entire API call in `try/except` — catch network failures and API errors separately

STEP 5 — Parse the JSON response **Phase 1 Week 3 reused**

- Extract the text from Gemini's response
- Strip any accidental markdown fences using `.strip()` and `.replace()`
- Parse using `json.loads()` into a Python list of dicts
- If parsing fails, raise `InvalidAIResponse` and print the raw response for debugging

STEP 6 — Print formatted terminal report **Phase 1 Week 1 + 2 reused**

- Use `enumerate()` to number each test case
- Print each test case in a clean structured format showing all fields
- Steps printed as a numbered sub-list

STEP 7 — Save to .json file **Phase 1 Week 3 reused**

- Save the parsed test cases to `generated_test_cases.json` using `json.dump()` with `indent=4`

- Print a confirmation message

File Structure You Need

```
project/
|
```

```
├── main.py                    # your script
├── user_story.txt             # user story you write manually
├── .env                        # GEMINI_API_KEY=your_key_here
├── generated_test_cases.json  # created by your script after running
└── requirements.txt           # google-generativeai, python-dotenv
```

What to Put in user_story.txt

```
As a registered user, I want to log in to the application
using my email and password so that I can access my dashboard.

Acceptance Criteria:
- Valid credentials should redirect to dashboard
- Invalid credentials should show an error message
- Empty fields should show a validation warning
- After 3 failed attempts, account should be locked
```

Expected AI Output Structure

Instruct Gemini to return **exactly this JSON structure** — put this in your system prompt:

```
[
  {
    "test_case_id": "TC001",
    "title": "Login with valid credentials",
    "preconditions": "User is registered and account is active",
    "steps": [
      "Open browser",
      "Navigate to login page",
      "Enter valid email",
      "Enter valid password",
      "Click login"
    ],
    "expected_result": "User is redirected to dashboard",
    "priority": "High",
    "status": "Not Run"
  }
]
```

Expected Terminal Output (Sample)

```
============================================
    AI-POWERED TEST CASE GENERATOR
============================================

[STEP 1] Loading API key from .env ... Done ✔
[STEP 2] Reading user story from user_story.txt ... Done ✔
[STEP 3] Building prompt ... Done ✔
[STEP 4] Calling Gemini API (temperature=0.2) ...

[STEP 5] Parsing AI response ... Done ✔
         → 5 test cases generated

============================================
         GENERATED TEST CASES REPORT
============================================

Test Case 1 : TC001
Title       : Login with valid credentials
Priority    : High
Precondition: User is registered and account is active
Steps       :
   1. Open browser
   2. Navigate to login page
   3. Enter valid email
   4. Enter valid password
   5. Click login
Expected    : User is redirected to dashboard
Status      : Not Run
--------------------------------------------
... (repeat for all 5 test cases)

[STEP 7] Saving test cases to generated_test_cases.json ... Done ✔

============================================
Total Generated : 5 | High : 2 | Medium : 2 | Low : 1
============================================
```

Concepts You're Practicing

Concept	Where It's Used	Phase / Week

`python-dotenv` , `.env` file	Loading API key safely	Phase 2 Week 2 Day 13
`google-generativeai` SDK	Calling Gemini API	Phase 2 Week 2 Day 8–9
`temperature=0.2` , `max_output_tokens`	Controlling AI output quality	Phase 2 Week 2 Day 10
System prompt engineering	Giving Gemini its QA role	Phase 2 Week 2 Day 11
File handling (`open` , `with`)	Reading `.txt` , writing `.json`	Phase 1 Week 3
`json.loads()` , `json.dump()`	Parsing AI response, saving output	Phase 1 Week 3
Custom exceptions (3 types)	`MissingAPIKey` , `EmptyUserStory` , `InvalidAIResponse`	Phase 1 Week 2
`try/except` (multiple layers)	API call, JSON parsing, file read	Phase 1 Week 2
`enumerate()`	Numbering test cases in report	Phase 1 Week 1
Priority counter	Counting High / Medium / Low at end	Phase 1 Week 1

Acceptance Criteria Checklist

#	Criteria
1	API key loaded from `.env` using `python-dotenv` — never hardcoded in script
2	`MissingAPIKey` custom exception raised if key is absent or empty
3	User story read from `user_story.txt` using file handling
4	`EmptyUserStory` custom exception raised if file is empty or missing
5	System prompt instructs Gemini to return raw JSON only — no markdown, no explanation
6	`temperature=0.2` and `max_output_tokens=1000` set explicitly
7	API call wrapped in `try/except` — network and API errors handled separately
8	Markdown fences stripped before `json.loads()` parsing
9	`InvalidAIResponse` raised if JSON parsing fails — raw response printed for debugging
10	Terminal report uses `enumerate()` and prints all fields per test case including numbered steps
11	Output saved to `generated_test_cases.json` with `indent=4`
12	Final summary line shows total generated + High / Medium / Low priority counts

Install Before You Start

```
pip install google-generativeai python-dotenv
```

.env file format:

```
GEMINI_API_KEY=your_actual_key_here
```

🔥 Why This Problem is Hard Enough

- **Three custom exceptions** across different failure points — not just one
- **Two layers of try/except** — one for API call, one for JSON parsing
- **AI output is unpredictable** — you must defensively strip and parse it — if your prompt is weak, Gemini won't return clean JSON and your parser will break
- **Priority counter** requires looping through parsed results and counting by field value
- **System prompt engineering** — the quality of your prompt directly controls the quality of your output

Notes before you start:

- Test each STEP in isolation — run STEP 1+2 first, confirm the file reads correctly, then add STEP 3+4, then STEP 5
- If Gemini returns markdown fences (```json), your `json.loads()` will fail — always strip before parsing
- JSONPlaceholder is gone — this is a real API call that costs tokens. Use a short user story to save quota during testing